How do you audit a computation you cannot see?

A treaty clause, a promise to a regulator and an API bill are the same claim. An outsider has two ways to test it: the bytes that came back, and the clock.

MACHINE ALIGNMENT TRANSPARENCY & SECURITY

Jiawei Li[1] · mentored by Gabriel Kulp[2] and Roy Rinberg[2]

[1]MATS 10.0 Scholar [2]MATS 10.0 Mentor Code + every run artifact: github.com/lijiawei20161002/inference-verification

THE AUDIT PROBLEM

Two governments agree to stop training something. Who checks — and with what?

Every compute-governance instrument ends in a promise about work done inside a building the auditor cannot enter. Nothing in the stack between a GPU and the outside world emits a statement about what the silicon actually did, so the auditor is left with whatever crosses the boundary.

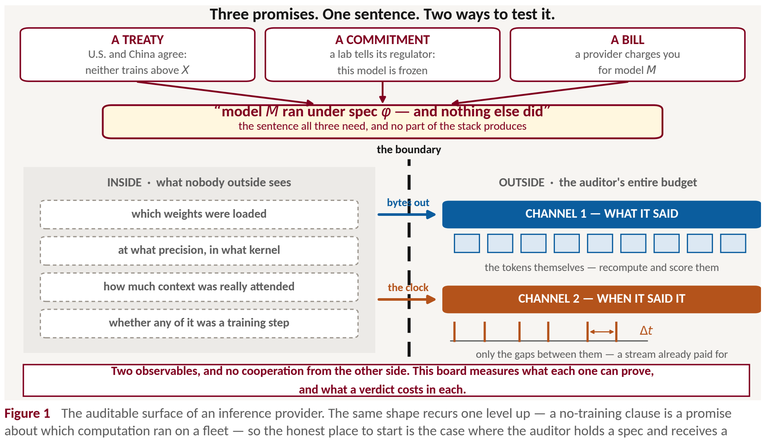

Figure 1 The auditable surface of an inference provider. The same shape recurs one level up — a no-training clause is a promise about which computation ran on a fleet — so the honest place to start is the case where the auditor holds a spec and receives a stream, and every claim can be measured on real hardware.

WHY THE PROMISE IS WORTH BREAKING

A 4-bit copy, a smaller sibling or a truncated context is pure margin, and invisible in the tokens the client pays for. But the deviations worth doing are exactly the ones that read fewer bytes — which is why they leave a second trace.

what the provider does instead	bytes it saves	worth it?
Serve Qwen3-0.6B as Qwen3-1.7B	2.83×	capability
4-bit (NF4) weights instead of bf16	2.55×	capacity
Attend half the contracted context	1.35×	capacity
Wrong seed / wrong temperature	1.00×	no

Table 1 Weight + KV bytes per decode step, from the timing grid's own byte counts (clock_channel.json, H100 PCIe, B=1). The last row is the control: a deviation that saves no bytes leaves no clock signature at all — and the two channels split exactly there.

THE TWO CHANNELS ARE BLIND IN OPPOSITE DIRECTIONS

	CHANNEL 1 — WHAT IT SAID	CHANNEL 2 — WHEN IT SAID IT
	Recompute M on the claimed tokens and read the per-token margin — or skip M and score against a cheap trusted proxy q.	Read the arrival times of a stream already bought. Zero audit FLOPs, zero cooperation from the provider, zero added latency.
sees	the sampler, the seed, the exact distribution; wholesale substitution at AUC 0.998 with no M	which positions were attended, and which parameters were read at prefill
blind	an fp8 KV cache — it moves the served distribution less than M's own rerun noise	anything reading the same bytes: wrong seed, wrong temperature, a top-k bug

Figure 2 Complementary by mechanism, not by accident: a deviation either changes the distribution the tokens were drawn from, or the bytes the GPU had to move, or both. One protocol scores both channels — standardized pAUC @ FPR ≤ 0.5%, held-out honest calibration, batch/pool ratio ≤ 10% enforced in code.

CHANNEL 1 · READ WHAT CAME BACK

One token proves almost nothing. A verdict is bought in bulk — and the price is fixed before you start.

Re-run M on the tokens the provider claims it produced and compare the per-token sampling margin. The honest and the cheating stream overlap almost completely: the audit rests on one number, the per-token separation d'.

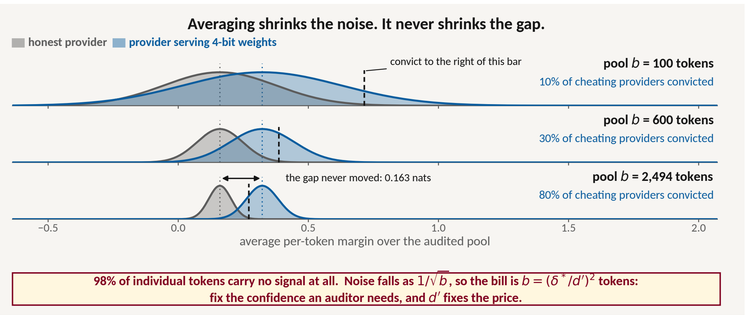

Figure 3 The pooling law with this experiment's own measured inputs — Qwen3-1.7B under 4-bit weights, per-token d' = 0.075 (cost_of_a_verdict_scores.npz). Against measured pAUC at 5 pool sizes on a 56,160-token pool the law holds to a mean absolute residual of 0.012 (pool_scaling.json) — which is what lets an auditor price a verdict before spending a budget.

WHAT CATCHES WHAT

Every detector is structurally blind to a different class of deviation, and which one wins is set by the attack, not the model. An auditor has to hold a portfolio.

	token DiFR	cross entropy	activation DiFR	token top-loc
4-bit weights	0.718	0.505	1.000	0.600
fp8 KV cache	0.511	0.507	1.000	0.517
wrong temperature	0.516	0.823	0.504	0.661
wrong seed	1.000	0.524	0.575	0.507
top-k bug (k=2)	0.553	0.510	0.499	0.523
top-k bug (k=32)	0.712	0.605	0.499	0.686

Table 2 headline_ratio.json. Re-scored from the same per-token scores as a previously published table that resampled from too small a pool, 16 of 24 cells fall, by a median of 0.137: an over-ratio pool does not exaggerate a signal, it invents one.

WHAT ONE VERDICT COSTS

Detectors are usually ranked by accuracy. An auditor buys neither an AUC nor a FLOP count — it buys tokens. Below: the cheapest honest verdict for each deviation, and what the same verdict costs if the provider is made to open up.

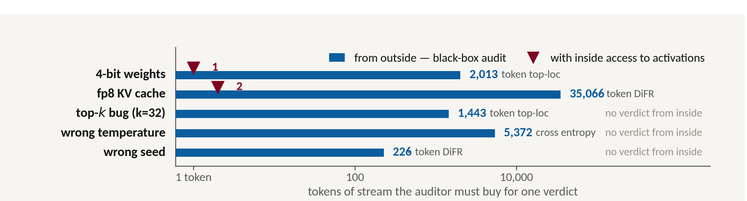

Figure 4 cost_of_a_verdict.json; Qwen3-1.7B, $\delta^* = 3.77$, 141 µs of H100 per recomputed token. Opening the provider up is worth 2,000× against quantization and nothing against a tampered sampler, which redraws from the honest distribution. Across the whole grid 14 of 35 cells have no price at all — and ranking by AUC hides every one of them.

CHANNEL 2 · READ WHEN IT CAME BACK

A token cannot arrive before its bytes have moved. The saving and the evidence are the same quantity.

One token at a time a GPU is memory-bound, not compute-bound: it must drag every weight it claims to be using across the bus before it can emit anything. Read fewer bytes and tokens arrive sooner — so the clock is evidence, at no audit FLOPs and no cooperation.

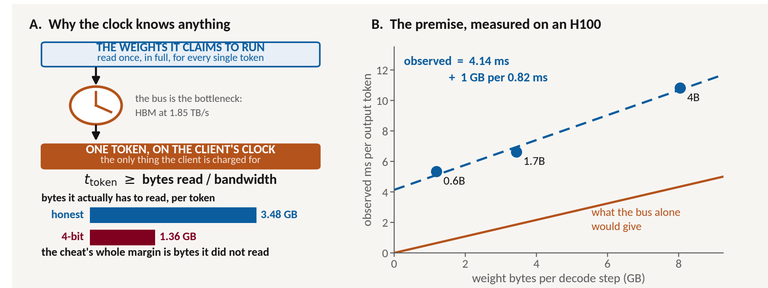

Figure 5 (B) clock_channel.json, H100 PCIe, CUDA graphs, B=1, one point per model. Time is linear in bytes — but at 66% of copy bandwidth, on a 4.14 ms software constant that is on no spec sheet. Under eager PyTorch the same GPU gives ~30 ms/token, flat in bytes: no channel at all.

EVERY OBVIOUS CLOCK TEST FAILS

what the premise seems to promise	what 126 timing cells say
Fewer bytes → proportionally less time	NF4 delivers 14% of it
4-bit weights are 3.9× faster, so 6 tokens convict	1.10×, and slower at B≥4
Half the KV bytes → half the slope	int4 KV is 1.6× steeper
Jitter is positive, so take the minimum gap	73% of honest gaps are 0 ms
One clock: the inter-token gap	prefill is 25,000× cheaper

Table 3 clock_channel.json (126 configurations) and clock_algos.json. The premise survives every row; the naive test survives none — and every failure attacks an absolute threshold.

THE TEST THAT SURVIVES: SUBTRACT

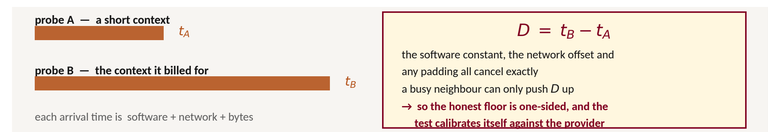

Figure 6 slope_verifier.json, 42 cells, same protocol as the token channel. A provider billing for 32,768 tokens of context and holding 512 is caught at pAUC 0.996 on 36 tokens of stream (8,192 → 62; half the context → 152) at 50 ms of wire jitter, against 2,013–35,066 for the token channel.

WHAT AN AUDITOR ACTUALLY GETS

A price, not a ranking: every verdict costed in tokens × seconds — and 14 of 35 cells cost infinity.
One protocol, one grid: 8 detectors × 6 deviations; under an enforced pool ceiling, 16 of 24 cells fall.
A second channel: the differential clock, 36–152 tokens of stream, and no cooperation from the provider.

None of this is a treaty regime — but it prices one. A clause that names a detector cannot be enforced; a clause that names a pool size, a false-alarm rate and a probe pair can be. The gaps are honest ones: a no-training clause is a claim about a fleet over months, not a stream over seconds; jitter here is device-side on one H100; models are 0.13B–8B, where the incentive to cheat is smallest; and no provider on this board adapts.

ACKNOWLEDGMENTS This work was carried out in the Machine Alignment, Transparency & Security (MATS) program, cohort 10.0. Thanks to Gabriel Kulp and Roy Rinberg for mentorship and to MATS for compute. Correspondence: jiawei2012220@gmail.com

REFERENCES [1] Karvonen et al. DiFR: Inference Verification Despite Nondeterminism, 2025 — the Tier-1 methodology, reimplemented clean-room here. [2] RoyRin, inference_verification_for_model_weight_exfil — the threat model. [3] Every number on this board regenerates from docs/results/ via paper/make_mats_poster.py.

Code Paper